

CURSO DE ENGENHARIA / TECNOLOGIA EM INTERNET DAS COISAS
DISCIPLINA DE SISTEMAS EMBARCADOS E IOT

RELATÓRIO TÉCNICO DE PROJETO FINAL

SISTEMA AUTOMATIZADO DE DIGITALIZAÇÃO 3D E FOTOGRAMETRIA

*Plataforma Mecatrônica de Precisão com Controle Cinemático por ESP32,
Interface IHM Analógica, Mira Óptica e Sustentabilidade por Upcycling*

Trabalho Final de Conclusão da Disciplina de IoT
Microcontrolador: ESP32 NodeMCU-32S (Dual-Core)
Data de Entrega: 05 de Outubro de 2026

1. DESCRIÇÃO GERAL DA APLICAÇÃO

1.1 Contextualização e Definição do Problema

A fotogrametria digital é uma técnica computacional e de sensoriamento óptico que permite reconstruir malhas tridimensionais densas (.OBJ / .STL) com texturas fotorrealistas a partir de um conjunto de fotografias bidimensionais sobrepostas de um objeto real. No entanto, o processo manual tradicional de captura apresenta três gargalos críticos que comprometem a convergência dos algoritmos de Structure-from-Motion (SfM):

- **1. Inconsistência no Espaçamento Angular:** A rotação manual do objeto introduz ângulos irregulares entre as fotos, gerando lacunas e perda de pontos homólogos na nuvem de pontos.
- **2. Vibrações e Desfoque de Movimento (Motion Blur):** A ausência de um tempo padronizado de amortecimento mecânico gera fotos borradas que inviabilizam o alinhamento de feixes.
- **3. Erros de Centralização Geométrica:** Objetos posicionados fora do centro exato de rotação sofrem distorções severas de paralaxe e oclusão de geometria.

O Sistema Automatizado de Digitalização 3D e Fotogrametria soluciona integralmente esses problemas através de uma bancada mecatrônica de alta precisão controlada pelo microcontrolador ESP32, integrando controle de micropassos calibrados (5120 passos/volta), mira laser colimada, display LCD 16x2 I2C, joystick analógico com auto-calibração no boot e sinalização audiovisual sincronizada.

1.2 Objetivos do Projeto

O projeto foi orientado pelos seguintes objetivos técnicos e acadêmicos:

- **Objetivo Geral:** Projetar, construir, programar e validar uma bancada mecatrônica automatizada de baixo custo para digitalização tridimensional padronizada de objetos por fotogrametria.
- **Controle Cinemático Calibrado:** Implementar acionamento em meio-passo suave (5120 passos/volta completa calibrados) com resolução de $0,0703^\circ$ por passo, garantindo fechamento perfeito de 360° sem folgas.
- **Sincronismo Audiovisual de Captura:** Integrar tempos de estabilização mecânica (800ms) com sinalização óptica (LEDs) e sonora (Buzzer) para disparo fotográfico preciso.
- **Algoritmo de Retorno Inteligente:** Desenvolver lógica cinemática de retorno à origem (0°) pelo caminho angular mais curto (Shortest Angular Path) em caso de cancelamento.
- **Alinhamento Óptico por Laser:** Incorporar mira laser colimada (reaproveitada de brinquedo Nerf) para centralização milimétrica do objeto antes do início do escaneamento, com desligamento automático durante as fotos.

1.3 Motivação e Sustentabilidade (Upcycling de E-Lixo)

- **Sustentabilidade e Upcycling Integrado:** Reaproveitamento duplo de materiais descartados: a estrutura mecânica e rolamento de precisão de um leitor óptico de CD/DVD, somado ao módulo de mira laser retirado de uma pistola de brinquedo Nerf.

- **Custo R\$ 0,00 em Novos Componentes:** Maximização total dos componentes eletrônicos do kit didático da disciplina (ESP32, motor 28BYJ-48, driver ULN2003, display I2C, 6 LEDs, buzzer, 1 protoboard 800 pontos e joystick), sem necessidade de novos custos financeiros.
- **Aplicação Prática:** Construção de uma ferramenta real e acessível para engenharia reversa, prototipagem rápida, impressão 3D, inspeção metrológica e preservação digital de peças.

2. DESENHO DA ARQUITETURA DO SISTEMA

2.1 Arquitetura em Camadas

O sistema foi estruturado em três camadas bem definidas para garantir modularidade, manutenibilidade e isolamento elétrico entre a lógica e a potência:

- **1. Camada de Aplicação e IHM:** Composta pelo Display LCD 16x2 I2C (exibição de telemetria, fotos e ângulos), Joystick analógico de 2 eixos, 6x LEDs de estado em topologia funil com resistores individuais de 300 Ω , Buzzer piezoelétrico e Diodo Laser de mira.
- **2. Camada de Processamento e Controle (ESP32):** Executa a Máquina de Estados Finitos (FSM), a auto-calibração dinâmica do centro neutro do joystick, a filtragem por sobre-amostragem (16x), o controle de micropassos calibrados (5120 passos) e o algoritmo do caminho angular mais curto.
- **3. Camada de Atuação Mecatrônica de Potência:** Composta pelo Driver ULN2003 (array de transistores Darlington) e o Motor de Passo 28BYJ-48 alimentados por barramento externo de 5V com GND compartilhado e desenergização térmica automática das bobinas.

2.2 Máquina de Estados Finitos (FSM)

O firmware opera sob uma máquina de estados finitos determinística com 4 modos principais:

- **Estado 1: MODO TESTE INICIAL:** Executado logo após a calibração no boot. Os LEDs ciclam suavemente e a mira laser acende. Transiciona para o Modo de Execução se o joystick for movido 3 vezes na mesma direção ou após o timeout de 60 segundos.
- **Estado 2: MENU DE EXECUÇÃO:** Permite ajustar a densidade de fotos com o Joystick para Esquerda (-4) ou Direita (+4), variando entre 8, 12, 16, 24 e 36 fotos. O Laser de mira permanece aceso para ajuste fino da peça. Dois toques para CIMA ($\blacktriangle \blacktriangle$) iniciam o scan.
- **Estado 3: EXECUTANDO SCAN 360°:** O laser é desligado automaticamente para não interferir na iluminação da foto. Para cada ângulo, a mesa gira, aguarda a pausa de estabilização (800ms), aciona o LED vermelho e emite o bip de captura. Dois toques para BAIXO ($\blacktriangledown \blacktriangledown$) pausam o sistema.
- **Estado 4: PAUSADO:** O motor é desenergizado e o LED amarelo acende com o laser reativado. Dois toques para CIMA ($\blacktriangle \blacktriangle$) despausam e continuam o scan. Dois toques para BAIXO ($\blacktriangledown \blacktriangledown$) cancelam e acionam o retorno inteligente ao ponto 0°.

2.3 Calibração Metrológica do Motor de Passo e Algoritmo do Caminho Mais Curto

Durante a validação prática na bancada, observou-se que o motor 28BYJ-48 operando com a constante teórica de 4096 passos deixava uma margem residual de aproximadamente 72° para fechar a volta completa. Realizou-se a calibração experimental da caixa de redução interna, estabelecendo a constante real de 5120 passos por volta completa (resolução de 0,0703°/passo), resultando em um fechamento angular de 360,00° exato.

Em caso de cancelamento da digitalização na pausa, o ESP32 calcula dinamicamente a menor distância angular para retornar à origem (0°):

- Se Passos Acumulados > 2560 (> 180°): O motor avança no sentido horário os passos restantes (5120 - Passos Acumulados).
- Se Passos Acumulados <= 2560 (<= 180°): O motor rebobina no sentido anti-horário os passos já dados (-Passos Acumulados).

Isso reduz o tempo de retorno em até 50% e evita estresse mecânico desnecessário.

3. RELAÇÃO DE PARTES E MATERIAIS (BOM)

A tabela a seguir discrimina todos os componentes empregados no protótipo final atualizado:

Item	Categoria	Componente / Especificação	Qtd.	Função no Sistema
1	Processamento	Placa ESP32 NodeMCU-32S (Xtensa Dual-Core 240MHz)	1 un.	Núcleo de controle em tempo real e IHM
2	Atuação	Motor de Passo Unipolar 28BYJ-48 (5V DC, Redução Calibrada)	1 un.	Tração angular com 5120 passos/volta completa
3	Potência	Módulo Driver ULN2003 (Array Darlington)	1 un.	Chaveamento de corrente das 4 bobinas
4	Display / IHM	Display LCD 16x2 com Módulo Adaptador I2C (PCF8574)	1 un.	Exibição de telemetria, fotos e ângulos
5	Entrada / IHM	Módulo Joystick Analógico 2 Eixos (10k)	1 un.	Navegação e comandos de controle

6	Mira Óptica	Módulo Diodo Laser Vermelho 650nm (Upcycling / Pistola Nerf)	1 un.	Alinhamento e centralização milimétrica da peça
7	Sinalização Sonora	Buzzer Piezoelétrico 5V	1 un.	Alertas sonoros de início, foto e fim de ciclo
8	Sinalização Óptica	6x LEDs 5mm (2 Verdes, 2 Amarelos, 2 Vermelhos em Funil)	6 un.	Indicação visual dos estados do sistema
9	Proteção	Resistores de Carbono 300 Ω (1/4W) [6 p/ LEDs + 1 p/ Laser]	7 un.	Limitação individual de corrente dos LEDs e do Laser
10	Montagem	Protoboard de 800 Pontos (Matriz Principal)	1 un.	Distribuição de sinais e barramentos de bancada
11	Mecânica	Estrutura Mecânica de Leitor de CD/DVD (Upcycling)	1 un.	Suporte giratório e rolamento de precisão
12	Conexões	Conjunto de Cabos Jumpers Flexíveis (MM, MF, FF)	Conj.	Interconexão elétrica dos circuitos
13	Alimentação	Fonte Externa 5V/2A + Cabo USB	1 un.	Alimentação de potência e lógica com GND comum

4. DIAGRAMA DE CONEXÃO DOS COMPONENTES ELETRÔNICOS

4.1 Pinout Geral do ESP32 NodeMCU-32S (30 Pinos)

A distribuição dos pinos no ESP32 foi projetada para evitar conflitos de barramento e garantir que os canais analógicos do joystick operem exclusivamente no bloco ADC1 (GPIOs 36 e 39), mantendo os pinos I2C dedicados no barramento padrão (GPIOs 21 e 22):

Componente	Pino do Módulo	Pino no ESP32	Descrição / Função
Display LCD 16x2	SDA	GPIO 21 (D21)	Linha de Dados I2C (Endereço 0x27)
Display LCD 16x2	SCL	GPIO 22 (D22)	Linha de Clock I2C
Display LCD 16x2	VCC / GND	VIN (5V) / GND	Alimentação do display e backlight
Driver ULN2003	IN1	GPIO 19 (D19)	Bobina A do Motor de Passo
Driver ULN2003	IN2	GPIO 18 (D18)	Bobina B do Motor de Passo
Driver ULN2003	IN3	GPIO 5 (D5)	Bobina C do Motor de Passo
Driver ULN2003	IN4	GPIO 17 (TX2)	Bobina D do Motor de Passo
Driver ULN2003	(+) / (-)	Fonte 5V / GND	Alimentação de potência externa com jumper ON
Joystick Analógico	VRX	GPIO 36 (VP)	Eixo Horizontal (Leitura ADC1_CH0)
Joystick Analógico	VRY	GPIO 39 (VN)	Eixo Vertical (Leitura ADC1_CH3)
Joystick Analógico	VCC / GND	3V3 / GND	Alimentação regulada de 3.3V para precisão
Mira Laser (Nerf)	(+) / (-)	GPIO 33 (D33) / GND	Controle óptico via resistor limitador de 300 Ω
LEDs Verdes (2x)	Anodo (+)	GPIO 26 (D26)	Sinalização de Pronto / Standby (com resistores de 300 Ω)
LEDs Amarelos (2x)	Anodo (+)	GPIO 27 (D27)	Sinalização de Rotação / Pausa (com resistores de 300 Ω)
LEDs Vermelhos (2x)	Anodo (+)	GPIO 14 (D14)	Sinalização de Captura de Foto (com resistores de 300 Ω)

Buzzer Piezo

(+) / (-)

GPIO 25 (D25) / GND

Emissão de tons PWM
de feedback sonoro

4.2 Topologia Elétrica e Barramento de GND Comum

Para eliminar ruídos induzidos pelas bobinas indutivas do motor e prevenir quedas de tensão na lógica do microcontrolador:

1. O Driver ULN2003 é energizado diretamente pela fonte de alimentação externa de 5V.
2. O polo negativo (GND) da fonte externa é interligado diretamente ao pino GND do ESP32, unificando a referência de terra de todo o circuito.
3. Os potenciômetros do joystick são alimentados pelo pino 3V3 do ESP32 para garantir estabilidade métrica no conversor analógico-digital (ADC1).

5. CÓDIGO-FONTE EMBARCADO COMPLETO

Abaixo é apresentado o código-fonte integral em C++ desenvolvido no ecossistema PlatformIO / Arduino Framework, contendo todas as rotinas de controle de micropassos calibrados (5120 passos/volta), máquina de estados, calibração dinâmica e desenergização térmica:

```
#include <Arduino.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

/*
 * =====
 * SISTEMA AUTOMATIZADO DE DIGITALIZAÇÃO 3D E FOTOGRAMETRIA
 * MODO AUTÔNOMO DE ALTA ESTABILIDADE (ZERO RISCO DE BROWNOUT / REINÍCIO)
 *
 * - 100% Imune a quedas de tensão e sobrecarga USB
 * - Modo Teste Inicial: Transição com 3x qualquer direção OU 1 minuto
 * - Menu de Execução: Ajuste de Fotos/Quadros (Esquerda: -4 | Direita: +4)
 * - 2x CIMA (▲▲): Iniciar Scan 360° com Pausa de Estabilização e Sinal de Disparo
 * - 2x BAIXO (▼▼): Pausar o movimento
 * - Na Pausa:
 *   * 2x CIMA (▲▲): Despausar e continuar
 *   * 2x BAIXO (▼▼): Cancelar com Cálculo de Menor Distância Angular até o Ponto 0
 * =====
 */

// ===== PINAGEM DO HARDWARE =====
const int PIN_JOY_X = 36; // GPIO36 (VP) - Eixo Horizontal
const int PIN_JOY_Y = 39; // GPIO39 (VN) - Eixo Vertical

const int PIN_BUZZER = 25; // Buzzer Piezoelétrico
const int PIN_LASER = 33; // Mira Laser (GPIO 33 / D33)

const int PIN_LED_VERDE = 26; // 2x LEDs Verdes
const int PIN_LED_AMAR = 27; // 2x LEDs Amarelos
const int PIN_LED_VERM = 14; // 2x LEDs Vermelhos

const int PIN_IN1 = 19; // Driver ULN2003 - IN1
const int PIN_IN2 = 18; // Driver ULN2003 - IN2
const int PIN_IN3 = 5; // Driver ULN2003 - IN3
const int PIN_IN4 = 17; // Driver ULN2003 - IN4 (TX2)

LiquidCrystal_I2C lcd(0x27, 16, 2);

// Matriz de meio-passo suave (5120 passos por volta completa - Calibrado)
const int PASSOS_VOLTA_COMPLETA = 5120;
```

```

const int passosMatriz[8][4] = {
  {1, 0, 0, 0}, {1, 1, 0, 0}, {0, 1, 0, 0}, {0, 1, 1, 0},
  {0, 0, 1, 0}, {0, 0, 1, 1}, {0, 0, 0, 1}, {1, 0, 0, 1}
};
int indicePasso = 0;

enum EstadoGlobal {
  MODO_TESTE_INICIAL,
  MENU_PRONTO_EXECUCAO,
  EXECUTANDO_SCAN,
  PAUSADO_SCAN
};
EstadoGlobal estadoAtual = MODO_TESTE_INICIAL;

enum ComandoJoy {
  JOY_NONE,
  JOY_UP,
  JOY_DOWN,
  JOY_LEFT,
  JOY_RIGHT
};

int totalPassos = 24;
int fotoAtual = 1;
long passosAcumulados = 0;
int anguloAtual = 0;

// Temporização de Inicialização
unsigned long tempoInicioBoot = 0;
const unsigned long TEMPO_LIMITE_TESTE_MS = 60000; // 1 minuto

// Detecção de 3 movimentos repetidos no modo teste
ComandoJoy ultimoComandoTeste = JOY_NONE;
int contagemComandosRepetidos = 0;
unsigned long tempoUltimoComandoTeste = 0;

// Detecção de Duplo Toque (2x CIMA / 2x BAIXO)
unsigned long tempoPrimeiroToque = 0;
int toquesCima = 0;
int toquesBaixo = 0;
const unsigned long JANELA_DUPLO_TOQUE = 600; // ms

// Calibração do Joystick
int centroJoyX = 2048;
int centroJoyY = 2048;

// Protótipos
void definirLeds(bool vermelho, bool amarelo, bool verde);
void desenergizarMotor();
void moverMotor(long passos, int delayMicros = 1300);
void bipCurto();
void bipFoto();
void bipFimCiclo();
int lerAnalogicoFiltrado(int pino);
void calibrarCentroJoystick();
ComandoJoy lerJoystick();
void ativarModoExecucao();
void atualizarTelaMenu();
bool checarDuploBaixoParaPausa(int tempoEsperaMs);
void gerenciarComandosPausa();
void retornarAoPontoZeroMenorDistancia();
void executarCicloLedsTeste();

void setup() {
  Serial.begin(115200);
  delay(200);

  tempoInicioBoot = millis();
  Serial.println("\n=====");
}

```

```

Serial.println(" SISTEMA DE DIGITALIZACAO 3D - MODO ALTA ESTABILIDADE ");
Serial.println("=====");

pinMode(PIN_BUZZER, OUTPUT);
pinMode(PIN_LASER, OUTPUT);
pinMode(PIN_LED_VERDE, OUTPUT);
pinMode(PIN_LED_AMAR, OUTPUT);
pinMode(PIN_LED_VERM, OUTPUT);

pinMode(PIN_IN1, OUTPUT);
pinMode(PIN_IN2, OUTPUT);
pinMode(PIN_IN3, OUTPUT);
pinMode(PIN_IN4, OUTPUT);

desenergizarMotor();
definirLeds(LOW, LOW, LOW);
digitalWrite(PIN_LASER, HIGH); // Liga laser de mira no boot para alinhamento

// Inicializa I2C e LCD
Wire.begin(21, 22);
lcd.init();
lcd.backlight();
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("SCANNER 3D IOT");
lcd.setCursor(0, 1);
lcd.print("Calibrando Joy..");

calibrarCentroJoystick();

lcd.clear();
lcd.setCursor(0, 0);
lcd.print("MODO DE TESTE");
lcd.setCursor(0, 1);
lcd.print("3x Joy ou 60 seg");
delay(800);
}

void loop() {
    ComandoJoy cmd = lerJoystick();

    // =====
    // ESTADO 1: MODO TESTE INICIAL (Com limite de 1 min ou 3 toques repetidos)
    // =====
    if (estadoAtual == MODO_TESTE_INICIAL) {
        digitalWrite(PIN_LASER, HIGH); // Mantém laser ativo para ajuste de mira

        // 1. Checa timeout de 1 minuto (60 segundos)
        unsigned long decorrido = millis() - tempoInicioBoot;
        if (decorrido >= TEMPO_LIMITE_TESTE_MS) {
            Serial.println(">> [MODO TESTE] 1 Minuto decorrido -> Entrando no Modo de Execucao!");
            ativarModoExecucao();
            return;
        }

        // 2. Checa se o usuário deu 3 toques seguidos para a mesma direção
        if (cmd != JOY_NONE) {
            if (cmd == ultimoComandoTeste && (millis() - tempoUltimoComandoTeste < 1200)) {
                contagemComandosRepetidos++;
                bipCurto();
                Serial.printf(">> [MODO TESTE] Toque repetido (%d/3)!\n", contagemComandosRepetidos);

                if (contagemComandosRepetidos >= 3) {
                    Serial.println(">> [MODO TESTE] 3 Toques confirmados -> Entrando no Modo de Execucao!");
                    ativarModoExecucao();
                    return;
                }
            } else {
                ultimoComandoTeste = cmd;
            }
        }
    }
}

```

```

        contagemComandosRepetidos = 1;
        bipCurto();
        Serial.println(">> [MOD0 TESTE] Primeiro toque registrado (1/3)");
    }
    tempoUltimoComandoTeste = millis();
}

// Reseta contador de toques se demorar mais de 1.2s
if (contagemComandosRepetidos > 0 && (millis() - tempoUltimoComandoTeste > 1200)) {
    contagemComandosRepetidos = 0;
    ultimoComandoTeste = JOY_NONE;
}

// Exibe contagem regressiva e status dos testes no LCD
static unsigned long ultimoCicloVisual = 0;
if (millis() - ultimoCicloVisual > 250) {
    ultimoCicloVisual = millis();
    int segundosRestantes = (TEMPO_LIMITE_TESTE_MS - decorrido) / 1000;

    lcd.setCursor(0, 0);
    lcd.print("TESTE: 3x -> EXEC");
    lcd.setCursor(0, 1);
    lcd.print("Auto: ");
    lcd.print(segundosRestantes);
    lcd.print("s (");
    lcd.print(contagemComandosRepetidos);
    lcd.print("/3) ");
}

// Ciclo suave de LEDs no modo teste
executarCicloLedsTeste();
return;
}

// =====
// ESTADO 2: MENU DE EXECUÇ O (CONFIGURA O DE FOTOS E DISPARO)
// =====
if (estadoAtual == MENU_PRONTO_EXECUCAO) {
    definirLeds(LOW, LOW, HIGH); // Verde aceso (Pronto para iniciar)
    digitalWrite(PIN_LASER, HIGH); // Mira laser acesa para posicionar a pe a no centro

    // Ajuste de fotos: Direita (+4) / Esquerda (-4)
    if (cmd == JOY_RIGHT) {
        if (totalPassos < 36) {
            totalPassos += 4;
            bipCurto();
            atualizarTelaMenu();
        }
    } else if (cmd == JOY_LEFT) {
        if (totalPassos > 8) {
            totalPassos -= 4;
            bipCurto();
            atualizarTelaMenu();
        }
    }
}

// 2 cliques seguidos para CIMA -> INICIA O SCAN 360 
if (cmd == JOY_UP) {
    toquesCima++;
    if (toquesCima == 1) {
        tempoPrimeiroToque = millis();
        Serial.println(">> [MENU] 1x CIMA recebido. Pressione CIMA novamente para Iniciar!");
    } else if (toquesCima >= 2) {
        toquesCima = 0;

        Serial.println(">> [MENU] 2x CIMA Confirmado! INICIANDO DIGITALIZAC O 360 ...");
        tone(PIN_BUZZER, 1800, 200);
        digitalWrite(PIN_LASER, LOW); // APAGA O LASER durante as fotos para n o estragar a textura 3D
        passosAcumulados = 0;
    }
}

```

```

        fotoAtual = 1;
        estadoAtual = EXECUTANDO_SCAN;
    }
}

if (toquesCima > 0 && (millis() - tempoPrimeiroToque > JANELA_DUPLO_TOQUE)) {
    toquesCima = 0;
}
}

// =====
// ESTADO 3: EXECUTANDO DIGITALIZAÇÃO 360°
// =====
else if (estadoAtual == EXECUTANDO_SCAN) {
    digitalWrite(PIN_LASER, LOW); // Garante que o laser fica apagado na captura das fotos

    while (fotoAtual <= totalPassos) {
        // 1. Calcula o próximo ângulo e passos
        long posicaoAlvoPassos = round((fotoAtual - 1) * ((float)PASSOS_VOLTA_COMPLETA / totalPassos));
        long passosParaGirar = posicaoAlvoPassos - passosAcumulados;

        if (passosParaGirar > 0) {
            definirLeds(LOW, HIGH, LOW); // Amarelo (Girando mesa)
            lcd.clear();
            lcd.setCursor(0, 0);
            lcd.print("GIRANDO MESA...");
            lcd.setCursor(0, 1);
            lcd.print("Passo: +");
            lcd.print(passosParaGirar);
            lcd.print(" stp");

            moverMotor(passosParaGirar, 1300);
            passosAcumulados += passosParaGirar;
            desenergizarMotor();
        }

        // 2. Atualiza o LCD
        anguloAtual = round((passosAcumulados / (float)PASSOS_VOLTA_COMPLETA) * 360.0);
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("FOTO: ");
        lcd.print(fotoAtual);
        lcd.print("/");
        lcd.print(totalPassos);

        lcd.setCursor(0, 1);
        lcd.print("ANG: ");
        lcd.print(anguloAtual);
        lcd.print((char)223);
        lcd.print(" [CAPTURA]");

        // 3. Pausa de estabilização mecânica (800ms) - Com monitoramento de pausa
        definirLeds(LOW, HIGH, LOW);
        if (!checarDuploBaixoParaPausa(800)) return;

        // 4. Sinal de Captura da Foto (LED Vermelho + Bip Fotográfico)
        definirLeds(HIGH, LOW, LOW); // Vermelho aceso (Momento da foto)
        bipFoto();
        Serial.printf(">> [CAPTURA FOTO %d/%d] Angulo: %d graus\n", fotoAtual, totalPassos, anguloAtual);

        // 5. Intervalo pós-disparo (1.5s) - Com monitoramento de pausa
        if (!checarDuploBaixoParaPausa(1500)) return;

        fotoAtual++;
    }

    // 6. Conclusão da Volta Completa de 360°
    if (passosAcumulados < PASSOS_VOLTA_COMPLETA) {
        long resto = PASSOS_VOLTA_COMPLETA - passosAcumulados;
    }
}

```

```

        moverMotor(resto, 1300);
        passosAcumulados += resto;
    }
    desenergizarMotor();

    bipFimCiclo();
    definirLeds(LOW, LOW, HIGH);
    digitalWrite(PIN_LASER, HIGH); // Reativa mira laser ao concluir

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("SCAN CONCLUIDO!");
    lcd.setCursor(0, 1);
    lcd.print("100% NO PONTO 0");
    delay(2500);

    passosAcumulados = 0;
    fotoAtual = 1;
    estadoAtual = MENU_PRONTO_EXECUCAO;
    atualizarTelaMenu();
}

// =====
// ESTADO 4: PAUSADO
// =====
else if (estadoAtual == PAUSADO_SCAN) {
    definirLeds(LOW, HIGH, LOW); // Amarelo aceso (Pausa)
    digitalWrite(PIN_LASER, HIGH); // Liga laser na pausa para checagem visual do objeto
    desenergizarMotor();
    gerenciarComandosPausa();
}
}

// ===== ATIVAÇÃO DO MODO DE EXECUÇÃO =====
void ativarModoExecucao() {
    Serial.println("\n>> [MODO DE EXECUCAO] Menu Ativado com Sucesso!");
    desenergizarMotor();
    definirLeds(LOW, LOW, HIGH);
    digitalWrite(PIN_LASER, HIGH); // Mira laser pronta para alinhamento

    tone(PIN_BUZZER, 1600, 150);
    toquesCima = 0;
    toquesBaixo = 0;
    estadoAtual = MENU_PRONTO_EXECUCAO;
    atualizarTelaMenu();
}

// ===== CHECAGEM DE 2x BAIXO PARA PAUSAR EM MOVIMENTO =====
bool checarDuploBaixoParaPausa(int tempoEsperaMs) {
    int fatias = tempoEsperaMs / 50;
    for (int i = 0; i < fatias; i++) {
        ComandoJoy cmd = lerJoystick();

        if (cmd == JOY_DOWN) {
            toquesBaixo++;
            Serial.printf(">> [EXECUCAO] Toque BAIXO para Pausa (%d/2)\n", toquesBaixo);
            if (toquesBaixo == 1) {
                tempoPrimeiroToque = millis();
            } else if (toquesBaixo >= 2) {
                toquesBaixo = 0;
                toquesCima = 0;
                tone(PIN_BUZZER, 800, 200);
                estadoAtual = PAUSADO_SCAN;

                lcd.clear();
                lcd.setCursor(0, 0);
                lcd.print(">> PAUSADO <<");
                lcd.setCursor(0, 1);
                lcd.print("^Cont | vvCanc");
            }
        }
    }
}

```

```

        Serial.println(">> [EXECUCAO] 2x BAIXO detectado -> SISTEMA PAUSADO!");
        return false;
    }
}

if (toquesBaixo > 0 && (millis() - tempoPrimeiroToque > 1200)) {
    toquesBaixo = 0;
}

delay(50);
}
return true;
}

// ===== GERENCIAMENTO DE COMANDOS QUANDO PAUSADO =====
void gerenciarComandosPausa() {
    ComandoJoy cmd = lerJoystick();

    // 2x CIMA: DESPAUSAR E CONTINUAR
    if (cmd == JOY_UP) {
        toquesCima++;
        Serial.printf(">> [PAUSA] Toque CIMA recebido (%d/2)\n", toquesCima);
        if (toquesCima == 1) {
            tempoPrimeiroToque = millis();
        } else if (toquesCima >= 2) {
            toquesCima = 0;
            toquesBaixo = 0;
            Serial.println(">> [PAUSA] 2x CIMA -> DESPAUSANDO E RETOMANDO SCAN!");
            tone(PIN_BUZZER, 1400, 150);
            estadoAtual = EXECUTANDO_SCAN;
            return;
        }
    }

    // 2x BAIXO: CANCELAR COM CÁLCULO DE MENOR DISTÂNCIA
    if (cmd == JOY_DOWN) {
        toquesBaixo++;
        Serial.printf(">> [PAUSA] Toque BAIXO recebido (%d/2)\n", toquesBaixo);
        if (toquesBaixo == 1) {
            tempoPrimeiroToque = millis();
        } else if (toquesBaixo >= 2) {
            toquesBaixo = 0;
            toquesCima = 0;
            Serial.println(">> [PAUSA] 2x BAIXO -> CANCELANDO SCAN COM CAMINHO MAIS CURTO!");
            tone(PIN_BUZZER, 500, 300);
            retornarAoPontoZeroMenorDistancia();

            fotoAtual = 1;
            estadoAtual = MENU_PRONTO_EXECUCAO;
            atualizarTelaMenu();
            return;
        }
    }

    if (toquesCima > 0 && (millis() - tempoPrimeiroToque > 1200)) {
        toquesCima = 0;
    }
    if (toquesBaixo > 0 && (millis() - tempoPrimeiroToque > 1200)) {
        toquesBaixo = 0;
    }
}

// ===== ALGORITMO DO CAMINHO MAIS CURTO AO PONTO ZERO =====
void retornarAoPontoZeroMenorDistancia() {
    if (passosAcumulados > 0) {
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("RESET PONTO 0...");
    }
}

```

```

    long passosRestantes = PASSOS_VOLTA_COMPLETA - passosAcumulados;

    // Se já passou da metade (> 180°), avança o resto no sentido horário
    if (passosAcumulados > (PASSOS_VOLTA_COMPLETA / 2)) {
        lcd.setCursor(0, 1);
        lcd.print("Avanco Rapido ->");
        Serial.printf(">> [RESET ZERO] Avanco rapido: +%ld passos\n", passosRestantes);
        moverMotor(passosRestantes, 1100);
    }
    // Se está na primeira metade (<= 180°), retrocede no sentido anti-horário
    else {
        lcd.setCursor(0, 1);
        lcd.print("<- Rebobinando");
        Serial.printf(">> [RESET ZERO] Rebobinando: -%ld passos\n", passosAcumulados);
        moverMotor(-passosAcumulados, 1100);
    }

    passosAcumulados = 0;
    desenergizarMotor();
}

// ===== INTERFACE DO MENU PRINCIPAL =====
void atualizarTelaMenu() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("MODO EXECUCAO :");
    lcd.setCursor(0, 1);
    lcd.print("FOTOS: ");
    lcd.print(totalPassos);
    lcd.print(" (^ GO)");
}

// ===== CICLO VISUAL DE LEDS NO MODO TESTE =====
void executarCicloLedsTeste() {
    static unsigned long tempoLed = 0;
    static int faseLed = 0;

    if (millis() - tempoLed > 400) {
        tempoLed = millis();
        faseLed = (faseLed + 1) % 4;

        switch (faseLed) {
            case 0: definirLeds(Low, Low, High); break; // Verde
            case 1: definirLeds(Low, High, Low); break; // Amarelo
            case 2: definirLeds(High, Low, Low); break; // Vermelho
            case 3: definirLeds(High, High, High); break; // Todos
        }
    }
}

// ===== LEITURA DO ANALÓGICO FILTRADA =====
int lerAnalogicoFiltrado(int pino) {
    long soma = 0;
    for (int i = 0; i < 16; i++) {
        soma += analogRead(pino);
        delayMicroseconds(100);
    }
    return soma / 16;
}

// ===== AUTO-CALIBRAÇÃO DO CENTRO =====
void calibrarCentroJoystick() {
    long sx = 0, sy = 0;
    for (int i = 0; i < 30; i++) {
        sx += analogRead(PIN_JOY_X);
        sy += analogRead(PIN_JOY_Y);
        delay(10);
    }
}

```

```

centroJoyX = sx / 30;
centroJoyY = sy / 30;
Serial.printf(">>> [CALIBRACAO] Centro: X=%d | Y=%d <<<\n", centroJoyX, centroJoyY);
}

// ===== DECODIFICADOR DO JOYSTICK (COM EIXOS CORRIGIDOS) =====
ComandoJoy lerJoystick() {
    static bool stickNeutro = true;

    int x = lerAnalogicoFiltrado(PIN_JOY_X);
    int y = lerAnalogicoFiltrado(PIN_JOY_Y);

    int deltaX = x - centroJoyX;
    int deltaY = y - centroJoyY;

    // Zona morta de 350 para alta sensibilidade em todas as direções
    const int ZONA_MORTA = 350;
    bool noCentro = (abs(deltaX) < ZONA_MORTA) && (abs(deltaY) < ZONA_MORTA);

    if (noCentro) {
        stickNeutro = true;
        return JOY_NONE;
    }

    if (stickNeutro) {
        stickNeutro = false; // Trava até soltar

        // Identifica o eixo predominante do movimento
        if (abs(deltaX) >= abs(deltaY)) {
            if (deltaX > ZONA_MORTA) {
                Serial.printf(">>> [JOYSTICK] BAIXO (dX=%+d, dY=%+d)\n", deltaX, deltaY);
                return JOY_DOWN;
            }
            if (deltaX < -ZONA_MORTA) {
                Serial.printf(">>> [JOYSTICK] CIMA (dX=%+d, dY=%+d)\n", deltaX, deltaY);
                return JOY_UP;
            }
        } else {
            if (deltaY > ZONA_MORTA) {
                Serial.printf(">>> [JOYSTICK] DIREITA (dX=%+d, dY=%+d)\n", deltaX, deltaY);
                return JOY_RIGHT;
            }
            if (deltaY < -ZONA_MORTA) {
                Serial.printf(">>> [JOYSTICK] ESQUERDA (dX=%+d, dY=%+d)\n", deltaX, deltaY);
                return JOY_LEFT;
            }
        }
    }

    return JOY_NONE;
}

// ===== CONTROLE DO MOTOR DE PASSO ULN2003 =====
void moverMotor(long passos, int delayMicros) {
    int direcao = (passos > 0) ? 1 : -1;
    long total = abs(passos);

    for (long i = 0; i < total; i++) {
        indicePasso += direcao;
        if (indicePasso >= 8) indicePasso = 0;
        if (indicePasso < 0) indicePasso = 7;

        digitalWrite(PIN_IN1, passosMatriz[indicePasso][0]);
        digitalWrite(PIN_IN2, passosMatriz[indicePasso][1]);
        digitalWrite(PIN_IN3, passosMatriz[indicePasso][2]);
        digitalWrite(PIN_IN4, passosMatriz[indicePasso][3]);

        delayMicroseconds(delayMicros);
    }
}

```

```

}

void desenergizarMotor() {
    digitalWrite(PIN_IN1, LOW);
    digitalWrite(PIN_IN2, LOW);
    digitalWrite(PIN_IN3, LOW);
    digitalWrite(PIN_IN4, LOW);
}

// ===== CONTROLE DOS LEDS =====
void definirLeds(bool vermelho, bool amarelo, bool verde) {
    digitalWrite(PIN_LED_VERM, vermelho);
    digitalWrite(PIN_LED_AMAR, amarelo);
    digitalWrite(PIN_LED_VERDE, verde);
}

void bipCurto() {
    tone(PIN_BUZZER, 1600, 40);
}

void bipFoto() {
    tone(PIN_BUZZER, 1800, 80);
}

void bipFimCiclo() {
    tone(PIN_BUZZER, 1000, 100);
    delay(120);
    tone(PIN_BUZZER, 1400, 100);
    delay(120);
    tone(PIN_BUZZER, 1800, 250);
}

```

6. ROTEIRO DE APRESENTAÇÃO E DEMONSTRAÇÃO (15 MINUTOS)

6.1 Planejamento do Tempo e Estrutura de Slides

Para cumprir rigorosamente o limite de 15 minutos com máxima clareza e impacto diante da banca avaliadora, a apresentação foi estruturada no seguinte formato:

- **Minuto 00:00 a 03:00 (Introdução e Motivação):** Apresentação da equipe, contextualização do problema da fotogrametria manual e demonstração do conceito de sustentabilidade e upcycling duplo (leitor de CD/DVD descartado + laser de brinquedo Nerf).
- **Minuto 03:00 a 06:00 (Arquitetura e Engenharia):** Explicação do diagrama de blocos, esquema elétrico de potência, controle em meio-passo calibrado de 5120 passos/volta e algoritmo do caminho mais curto.
- **Minuto 06:00 a 12:00 (Demonstração Prática ao Vivo na Bancada):** Execução prática do protótipo ligando o sistema, calibrando, alinhando com a mira laser, ajustando fotos no LCD, iniciando o scan e demonstrando a pausa e o cancelamento inteligente.
- **Minuto 12:00 a 15:00 (Resultados e Conclusão):** Exibição do modelo 3D reconstruído em software de fotogrametria e considerações finais.

6.2 Script da Demonstração Prática ao Vivo na Bancada

- **1. Inicialização e Calibração:** "Ao ligar a bancada, o ESP32 calibra dinamicamente o centro neutro do joystick e entra em modo de teste com os LEDs ciclando."

- **2. Transição e Mira Laser:** "Damos 3 toques no joystick para entrar no Modo de Execução. Notem que a mira laser acende automaticamente, permitindo alinhar e centralizar a peça com precisão milimétrica no centro do prato giratório."
- **3. Ajuste de Resolução no Display:** "Com o joystick para esquerda ou direita, escolhemos a quantidade exata de fotos no LCD (ex: 24 fotos para uma peça com detalhes médios)."
- **4. Execução do Scan 360°:** "Dois toques para CIMA iniciam o scan. O laser apaga automaticamente para não poluir a foto, a mesa rotaciona suavemente com os 5120 passos calibrados, estabiliza as vibrações mecânicas e o LED vermelho com o bip sinalizam o momento exato de captura da imagem."
- **5. Pausa e Cancelamento Inteligente:** "Durante a rotação, damos dois toques para BAIXO para pausar. Ao cancelar, o ESP32 calcula o caminho mais curto até a origem (0°) e desenergiza o motor imediatamente para evitar aquecimento."